

Product Review And Rating Management

Pepperfry furniture E-Commerce Database Management System

1. Database Design

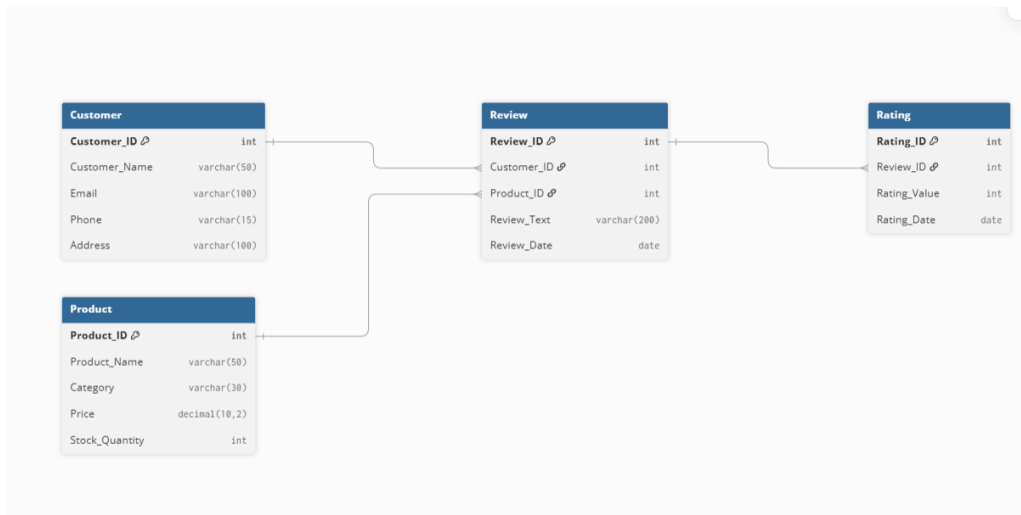
1.1 Review Table

Attribute	Data Type	Key Type
Review_ID	NUMBER	Primary Key
Customer_ID	NUMBER	Foreign Key
Product_ID	NUMBER	Foreign Key
Review_Text	VARCHAR2(200)	-
Review_Date	DATE	-

1.2 Rating Table

Attribute	Data Type	Key Type
Rating_ID	NUMBER	Primary Key
Review_ID	NUMBER	Foreign Key
Rating_Value	NUMBER(1)	-
Rating_Date	DATE	-

2. Table Relationship



3. Table Creation

3.1 Create Review Table

Creates a table to store customer reviews for products.

```
SQL> CREATE TABLE Review
 2  (
 3      Review_ID NUMBER PRIMARY KEY,
 4      Customer_ID NUMBER,
 5      Product_ID NUMBER,
 6      Review_Text VARCHAR2(200),
 7      FOREIGN KEY (Customer_ID)
 8      REFERENCES Customer(Customer_ID),
 9      FOREIGN KEY (Product_ID)
10      REFERENCES Product(Product_ID)
11  );

Table created.
```

3.2 Create Rating Table

Creates a table to store product ratings.

```
SQL> CREATE TABLE Rating
 2  (
 3      Rating_ID NUMBER PRIMARY KEY,
 4      Review_ID NUMBER,
 5      Rating_Value NUMBER(1) CHECK (Rating_Value BETWEEN 1 AND 5),
 6      Rating_Date DATE,
 7      FOREIGN KEY (Review_ID)
 8          REFERENCES Review(Review_ID)
 9  );

Table created.
```

4. Review Data Insertion

Inserts customer review records into the Review table

```
SQL> INSERT INTO Review
 2  VALUES (1, 1, 1, 'Excellent sofa with good quality',
 3  TO_DATE('01-09-2026', 'DD-MM-YYYY'));

1 row created.

SQL>
SQL> INSERT INTO Review
 2  VALUES (2, 2, 2, 'Comfortable and stylish chair',
 3  TO_DATE('03-09-2026', 'DD-MM-YYYY'));

1 row created.

SQL>
SQL> INSERT INTO Review
 2  VALUES (3, 3, 3, 'Good quality dining table',
 3  TO_DATE('05-09-2026', 'DD-MM-YYYY'));

1 row created.

SQL>
SQL> INSERT INTO Review
 2  VALUES (4, 4, 4, 'Nice wooden bed',
 3  TO_DATE('07-09-2026', 'DD-MM-YYYY'));

1 row created.
```

5. Rating Data Insertion

Inserts rating values for the product reviews.

```

SQL> INSERT INTO Rating VALUES
  2  (1, 1, 5, TO_DATE('01-09-2026', 'DD-MM-YYYY'));

1 row created.

SQL>
SQL> INSERT INTO Rating VALUES
  2  (2, 2, 4, TO_DATE('03-09-2026', 'DD-MM-YYYY'));

1 row created.

SQL>
SQL> INSERT INTO Rating VALUES
  2  (3, 3, 5, TO_DATE('05-09-2026', 'DD-MM-YYYY'));

1 row created.

SQL>
SQL> INSERT INTO Rating VALUES
  2  (4, 4, 3, TO_DATE('07-09-2026', 'DD-MM-YYYY'));

1 row created.

SQL>
SQL> INSERT INTO Rating VALUES
  2  (5, 5, 4, TO_DATE('09-09-2026', 'DD-MM-YYYY'));

1 row created.

```

6. Retrieve Product Review Details

Retrieves product-wise customer review information

```

SQL> SELECT
  2     P.Product_ID,
  3     P.Product_Name,
  4     R.Review_ID,
  5     R.Customer_ID,
  6     R.Review_Text,
  7     R.Review_Date
  8 FROM Product P
  9 JOIN Review R
 10 ON P.Product_ID = R.Product_ID;

```

PRODUCT_ID	PRODUCT_NAME	REVIEW_ID	CUSTOMER_ID	REVIEW_TEXT	REVIEW_DA
1	Sofa	1	1		

PRODUCT_ID	PRODUCT_NAME	REVIEW_ID	CUSTOMER_ID	REVIEW_TEXT	REVIEW_DA
				Excellent sofa with good quality	01-SEP-26

7. Calculate Average Product Ratings

Calculates the average rating of each product using aggregate functions

```
SQL> SELECT
  2     P.Product_ID,
  3     P.Product_Name,
  4     AVG(RT.Rating_Value) AS Average_Rating
  5 FROM Product P
  6 JOIN Review R
  7   ON P.Product_ID = R.Product_ID
  8 JOIN Rating RT
  9   ON R.Review_ID = RT.Review_ID
 10 GROUP BY
 11     P.Product_ID,
 12     P.Product_Name
 13 ORDER BY
 14     P.Product_ID;
```

```
PRODUCT_ID
-----
PRODUCT_NAME
-----
AVERAGE_RATING
-----
```

```
1
```

```
Sofa
```

```
5
```

```
2
```

```
Dining Table
```

```
4
```

```
PRODUCT_ID
-----
```

```
PRODUCT_NAME
-----
```

```
AVERAGE_RATING
-----
```

```
3
```

```
Office Chair
```

```
5
```

```
4
```

```
Bed
```

8. Identify Highly Rated Products

Identifies products with an average rating of 4 or above.

```

SQL> SELECT
  2     P.Product_ID,
  3     P.Product_Name,
  4     AVG(RT.Rating_Value) AS Average_Rating
  5 FROM Product P
  6 JOIN Review R
  7     ON P.Product_ID = R.Product_ID
  8 JOIN Rating RT
  9     ON R.Review_ID = RT.Review_ID
10 GROUP BY
11     P.Product_ID,
12     P.Product_Name
13 HAVING AVG(RT.Rating_Value) >= 4
14 ORDER BY Average_Rating DESC;

```

```
PRODUCT_ID
```

```
PRODUCT_NAME
```

```
AVERAGE_RATING
```

```
-----
```

```
1
```

```
Sofa
```

```
5
```

```
6
```

```
Coffee Table
```

```
5
```

```
PRODUCT_ID
```

```
PRODUCT_NAME
```

```
AVERAGE_RATING
```

```
-----
```

```
3
```

```
Office Chair
```

```
5
```

```
9
```

```
Study Table
```

9. Product Rating Report

Generates a complete product-wise rating summary using aggregate functions

```

SQL> SELECT
2     P.Product_ID,
3     P.Product_Name,
4     COUNT(RT.Rating_ID) AS Total_Ratings,
5     AVG(RT.Rating_Value) AS Average_Rating,
6     MIN(RT.Rating_Value) AS Minimum_Rating,
7     MAX(RT.Rating_Value) AS Maximum_Rating
8 FROM Product P
9 JOIN Review R
10      ON P.Product_ID = R.Product_ID
11 JOIN Rating RT
12      ON R.Review_ID = RT.Review_ID
13 GROUP BY
14     P.Product_ID,
15     P.Product_Name
16 ORDER BY
17     P.Product_ID;

```

PRODUCT_ID	PRODUCT_NAME	TOTAL_RATINGS	AVERAGE_RATING	MINIMUM_RATING	MAXIMUM_RATING
1	Sofa	1	5	5	5
2	Dining Table	1	4	4	4
3	Office Chair	1	5	5	5
4					